

6CS01: Artificial Intelligence and Machine Learning

Part II - Language Task Report

Name: Nischal Man Singh

Group: L6CG5

Student ID: 2414202

Tutor: Bijaya Ghimire

Module Leader: Siman Giri

Academic Year	Module	Assessment Number	Assessment Type
2026	Artificial Intelligence and Machine Learning	1	Report

1. Title

Sentiment Analysis of Tweets using RNN, LSTM, and Word2Vec Embeddings

2. Abstract

This project investigates sentiment classification of short social media text (tweets) into sentiment categories (e.g., positive, negative, and neutral). Automatic sentiment analysis is important for understanding public opinion at scale, enabling applications such as brand monitoring, product feedback mining, crisis detection, and policy analysis. We developed and compared two sequence-based deep learning approaches: a Simple Recurrent Neural Network (RNN) and a Long Short-Term Memory (LSTM) network. To represent words numerically, we evaluated (i) randomly initialized trainable embeddings learned during model training and (ii) pretrained Word2Vec embeddings to capture semantic similarity and reduce the need for large labeled datasets. Models were trained using supervised learning with cross-entropy loss and optimized with Adam. Performance was evaluated using accuracy and class-wise precision, recall, and F1-score derived from the confusion matrix. Overall, the LSTM achieved more stable convergence and better generalization than the Simple RNN, reflecting LSTM's ability to mitigate vanishing-gradient issues in long sequences. Incorporating pretrained Word2Vec embeddings further improved performance compared to random embeddings by providing richer word representations. The results suggest that LSTM + Word2Vec is a strong baseline for tweet sentiment classification, and future improvements can be achieved through hyperparameter tuning, regularization, and larger or more balanced datasets.

3. Introduction

Text classification is a core Natural Language Processing (NLP) task in which an input text sequence is assigned to one of several predefined categories. In sentiment analysis, the goal is to identify the emotional polarity expressed in text, such as classifying tweets into positive, negative, and neutral sentiment. Tweets are short, noisy, and often contain informal language, emojis, abbreviations, URLs, hashtags, and user mentions. These characteristics make sentiment classification challenging, yet highly valuable for real-world decision-making.

Traditional machine learning approaches rely on manually engineered features (e.g., bag-of-words, TF-IDF). Deep learning methods, particularly Recurrent Neural Networks (RNNs), are better suited for sequence data because they model word order and contextual dependencies. However, vanilla RNNs suffer from vanishing gradients when sequences become longer, limiting their ability to capture long-range context. Long Short-Term Memory (LSTM) networks address this limitation using gating mechanisms that regulate information flow and preserve useful context over longer time steps.

Another critical component in text classification is word representation. Word embeddings map tokens into dense vectors that capture semantic relationships. Word2Vec embeddings, trained on large unlabeled corpora, often improve performance by providing strong initial word representations compared to randomly initialized embeddings.

4. Dataset

4.1 Source

The dataset used in this project consists of labeled Twitter posts (tweets) collected for the purpose of sentiment classification. Each tweet in the dataset is annotated with a sentiment label representing the emotional tone expressed in the text. The dataset may be obtained from publicly available sources such as the Sentiment140 dataset or a custom-built dataset created through manual annotation. The sentiment labels generally belong to three categories: positive, negative, and neutral.

The dataset is specifically designed for Natural Language Processing (NLP) tasks involving short and informal social media text. Since tweets often contain abbreviations, emojis, hashtags, mentions, and noisy textual patterns, the dataset provides a realistic environment for evaluating deep learning-based sentiment analysis models.

4.2 Data Size

The dataset contains a total of [FILL FROM ATTACHMENT] tweet samples used for training and evaluating the models. The sentiment classes included in the dataset are Positive, Negative, and Neutral. The distribution of samples across these classes is represented as [FILL CLASS DISTRIBUTION], which helps determine whether the dataset is balanced or imbalanced.

Each tweet is transformed into a sequence of numerical tokens representing words or subwords. These sequences are padded or truncated to a fixed maximum sequence length so that they can be processed efficiently by neural network architectures such as RNNs and LSTMs. The primary input features of the models are therefore fixed-length tokenized text sequences.

4.3 Preprocessing

Raw tweets collected from social media platforms often contain substantial noise and inconsistencies. Therefore, several preprocessing techniques were applied to standardize the text and improve model performance.

First, all tweet text was converted into lowercase format to reduce vocabulary duplication caused by capitalization differences. URLs, HTML entities, and unnecessary textual artifacts were removed because they do not contribute meaningful sentiment information. Twitter-specific patterns such as user mentions (@username) and hashtags (#topic) were either removed entirely or converted into clean keyword forms.

Punctuation marks, special symbols, and excessive whitespace were also removed to simplify textual structure. The cleaned tweets were then tokenized into individual words or subword units. Optional preprocessing techniques such as stopword removal and stemming/lemmatization were considered to further reduce noise and normalize vocabulary.

Finally, because neural networks require inputs of consistent size, all token sequences were padded or truncated to a predefined maximum sequence length LLL. This preprocessing pipeline ensured that the textual data became structured, standardized, and suitable for deep learning models.

5. Methodology

5.1 Text Preprocessing Pipeline

The preprocessing pipeline consisted of several sequential steps designed to convert raw tweet text into machine-readable numerical representations. Initially, noisy elements such as URLs, mentions, hashtags, emojis, and unnecessary symbols were cleaned from the raw text. The cleaned tweets were then tokenized into individual words.

A vocabulary dictionary was constructed by assigning a unique integer index to each word appearing in the dataset. Each tweet was subsequently transformed into a sequence of these integer indices. Since tweets naturally vary in length, padding and truncation techniques were used to ensure uniform sequence size across all samples. Finally, the dataset was divided into training, validation, and testing subsets to facilitate unbiased model evaluation.

5.2 Model Architectures

5.2.1 Simple RNN

The Simple Recurrent Neural Network (RNN) was implemented as a baseline sequential learning model for sentiment classification. The architecture began with an embedding layer that converted sparse integer tokens into dense vector representations. These embeddings were either randomly initialized and trained during learning or initialized using pretrained Word2Vec vectors.

The embedding output was then passed through a SimpleRNN layer containing hidden units h_{t-1} , which allowed the model to process sequential dependencies within tweet text. Optional dropout regularization was included to reduce overfitting during training. Finally, a dense output layer with either sigmoid activation (binary classification) or softmax activation (multiclass classification) generated the sentiment predictions.

Although Simple RNNs are capable of handling sequential data, they often struggle with long-term dependencies because of the vanishing gradient problem. As a result, their ability to capture contextual relationships across longer sequences is limited.

5.2.2 LSTM

To overcome the limitations of traditional RNNs, a Long Short-Term Memory (LSTM) architecture was implemented. Similar to the RNN model, the LSTM architecture began with an embedding layer for transforming words into dense vector representations. The embedding outputs were then passed into an LSTM layer containing hidden units h_{t-1} .

Unlike standard RNNs, LSTMs use specialized memory cells and gating mechanisms, including input, forget, and output gates, to selectively preserve or discard information over time. This allows the model to retain long-range contextual information and significantly improves gradient flow during training. Dropout and recurrent dropout regularization techniques were optionally applied to improve generalization performance. The final dense output layer produced sentiment class predictions using sigmoid or softmax activation functions depending on the classification setup.

5.2.3 Word2Vec Embedding Integration

Word embeddings play a crucial role in capturing semantic meaning in text data. In this project, pretrained Word2Vec embeddings were integrated into both the RNN and LSTM architectures. Word2Vec provides dense vector representations in which semantically similar words occupy nearby positions in vector space.

The pretrained vectors were loaded and mapped to the dataset vocabulary. These vectors were then used to initialize the embedding layer weights. Two embedding strategies were explored:

Frozen embeddings, where the pretrained vectors remained unchanged during training.

Fine-tuned embeddings, where embedding weights were updated during model training to better adapt to the dataset.

Using pretrained embeddings generally improves model generalization, particularly when training data is limited.

5.3 Loss Function

The loss function measures the discrepancy between predicted outputs and actual sentiment labels. For binary sentiment classification tasks, Binary Cross-Entropy loss was used. For multiclass classification involving positive, negative, and neutral sentiments, Sparse Categorical Cross-Entropy or Categorical Cross-Entropy was employed.

These loss functions are well-suited for classification tasks because they penalize incorrect predictions and guide the optimization process toward minimizing classification error.

5.4 Optimizer

The Adam optimizer was used for model training due to its computational efficiency and adaptive learning rate capabilities. Adam combines the advantages of momentum and RMSProp optimization techniques, enabling faster convergence and stable training performance.

The optimizer learning rate α was carefully selected to balance convergence speed and training stability.

Hyperparameter selection significantly affects training stability, convergence behavior, and overall classification performance.

6. Experiments and Results

6.1 RNN vs. LSTM Performance

Both the Simple RNN and LSTM models were trained and evaluated under identical experimental conditions, including the same preprocessing pipeline, train-validation-test splits, optimizer settings, and evaluation metrics.

The experimental results generally indicate that the LSTM model achieved superior performance compared to the Simple RNN. The improved performance is primarily attributed to the LSTM's gating mechanisms, which allow it to preserve long-term dependencies and mitigate the vanishing gradient problem.

If the results demonstrate significantly higher accuracy and lower validation loss for the LSTM, this supports the theoretical advantages of LSTM architectures for sequential text processing tasks. However, if the performance difference is minimal, possible explanations may include short sequence lengths, limited training samples, insufficient hyperparameter tuning, or dataset simplicity.

6.2 Computational Efficiency

Computational efficiency was evaluated using a consistent hardware environment such as Google Colab CPU or GPU runtime. The comparison focused on training time, memory consumption, and overall computational cost.

The results show that LSTM networks generally require more computational resources than Simple RNNs because of their additional memory cells and gating operations. Consequently, LSTMs consume more memory and require longer training time per epoch.

Despite the increased computational cost, LSTMs typically provide more stable learning and higher predictive performance, making them more suitable for complex sentiment analysis tasks.

6.3 Training with Different Embeddings

The experimental findings suggest that pretrained Word2Vec embeddings improved sentiment classification performance compared to randomly initialized embeddings. Since Word2Vec embeddings already encode semantic and contextual relationships between words, the models were able to generalize better and converge more efficiently during training.

The improvement was especially noticeable in situations involving limited labeled data or linguistically diverse tweet content.

6.4 Model Evaluation

Model performance was evaluated using several classification metrics to provide a comprehensive understanding of predictive quality.

Accuracy

Accuracy measures the overall proportion of correctly classified tweets among all predictions.

Confusion Matrix

The confusion matrix provides a detailed breakdown of prediction performance across sentiment classes by comparing true labels with predicted labels. This helps identify common misclassification patterns.

Precision, Recall, and F1-Score

Precision measures the correctness of positive predictions, recall measures the ability to identify actual positive samples, and the F1-score provides a harmonic balance between precision and recall. These metrics are especially important for imbalanced datasets.

Overall, the evaluation results demonstrate that the LSTM model achieved more reliable and balanced performance across all sentiment classes compared to the Simple RNN baseline.

7. Conclusion and Future Work

This project explored the application of recurrent neural network architectures for tweet sentiment classification. A comparative analysis between Simple RNN and LSTM models revealed that LSTM generally provides superior predictive performance, improved convergence behavior, and greater stability during training. These improvements are mainly due to the LSTM's ability to preserve long-term contextual dependencies through specialized memory gating mechanisms.

The study also demonstrated the effectiveness of pretrained Word2Vec embeddings in enhancing semantic representation and improving classification accuracy. Models initialized with Word2Vec embeddings consistently outperformed those using randomly initialized embeddings, highlighting the importance of meaningful vector representations in Natural Language Processing tasks.

Despite achieving promising results, several opportunities remain for future improvement.

Hyperparameter optimization techniques such as grid search or Bayesian optimization could be employed to further improve model performance. Additional regularization methods, including stronger dropout configurations, early stopping, and L2 weight decay, may help reduce overfitting.

Future work could also involve advanced preprocessing strategies such as emoji interpretation, sarcasm detection, negation handling, and contextual sentiment analysis. Furthermore, more sophisticated deep learning architectures such as Bidirectional LSTMs (BiLSTMs), CNN-LSTM hybrids, attention-based models, and Transformer-based architectures like BERT could be explored to achieve higher sentiment classification accuracy and better contextual understanding.